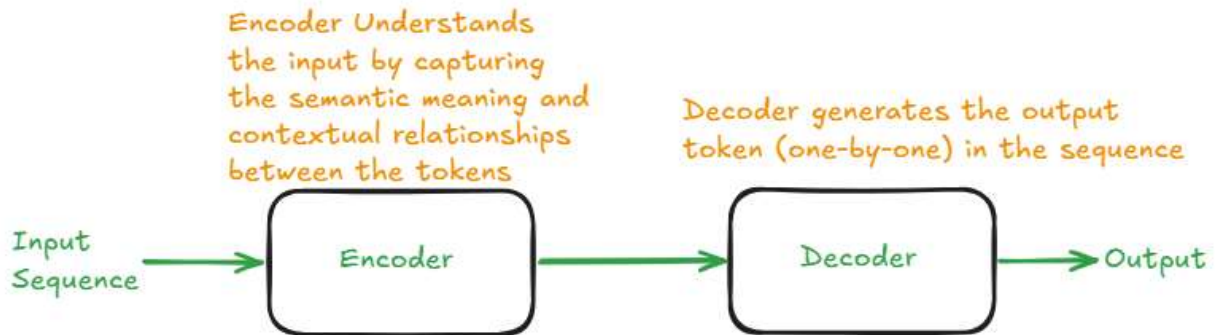


Transformer Architecture

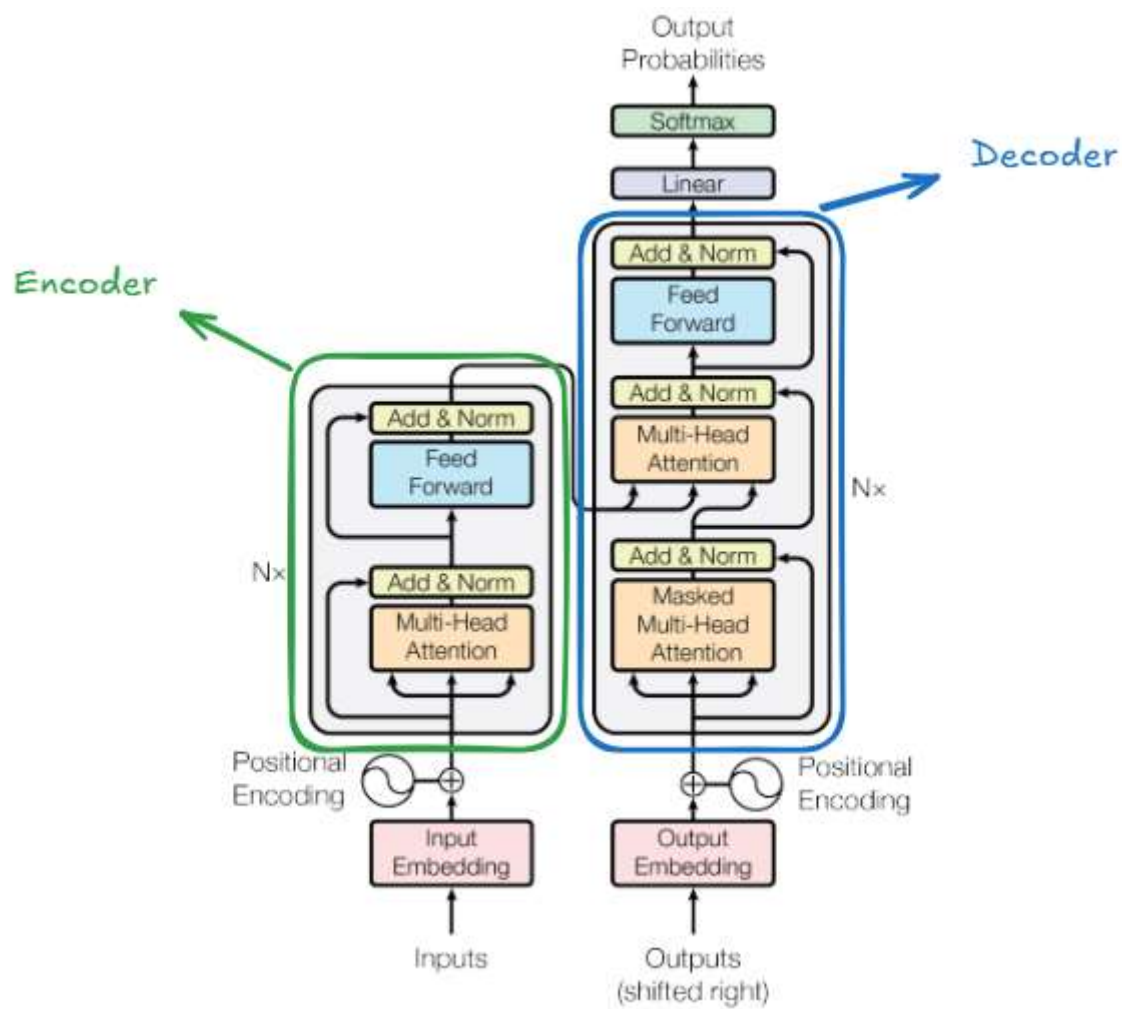
A Transformer model consists of two main parts:

1. **Encoder**
2. **Decoder**

Below Figure shows the description of **Encoder** and **Decoder** of the Transformer



The Architecture of **Encoder** and **Decoder** of the Transformer is shown in the below Figure



Consider an input sequence

My name is Ahsan and I love computer vision

Step 1: Tokenization



Step 2: Embeddings

Embeddings are vectors of floating-point numbers which represents the meaning of the token.

For creating Embeddings of tokens, we have to provide the Embedding dimension.

The Embedding dimensions in Transformer architecture is denoted by d_{model}

If $d_{model} = 8$, then the Embeddings for each token are represented as:

Token ID	8-Dimensional Embedding Vector
101	[0.12, -0.45, 0.88, 0.33, -0.21, 0.76, 0.55, -0.09]
9	[0.44, 0.11, -0.67, 0.29, 0.91, -0.38, 0.10, 0.72]
18	[-0.56, 0.48, 0.23, -0.11, 0.77, 0.65, -0.42, 0.18]
20	[0.31, -0.88, 0.59, 0.47, -0.36, 0.92, 0.14, -0.27]
8	[-0.15, 0.67, -0.92, 0.81, 0.24, -0.55, 0.38, 0.49]
200	[0.73, -0.22, 0.41, -0.64, 0.58, 0.17, -0.89, 0.26]
202	[-0.34, 0.95, 0.12, -0.48, 0.69, -0.71, 0.53, 0.07]
300	[0.28, 0.36, -0.74, 0.93, -0.16, 0.44, 0.61, -0.52]
1012	[-0.81, 0.25, 0.57, -0.33, 0.46, 0.79, -0.08, 0.91]

Thus, for the given input sequence (which contains 9 tokens). The output of **Input Embedding Layer** contains the shape (9, 8).

*Note: For real world data there are large number of input token for e.g. 50,000 token and the d_{model} is set to 256 or 512 or 768 or 1024 etc. The output of **Input Embedding Layer** contains the shape (50000, 512) $\rightarrow$ if $d_{model}=512$*

Step 3: Positional Encoding

In RNN, the tokens in the sequence are feed one-by-one to the input of model. So the model understand the position of every token in the input sequence.

Unlike RNN, in Transformer all the tokens are feed to the input of Encoder at the same time. So the Encoder does not understand the position of tokens in the input sequence.

Positional encoding adds information about:

- token position
- word order
- sequence structure

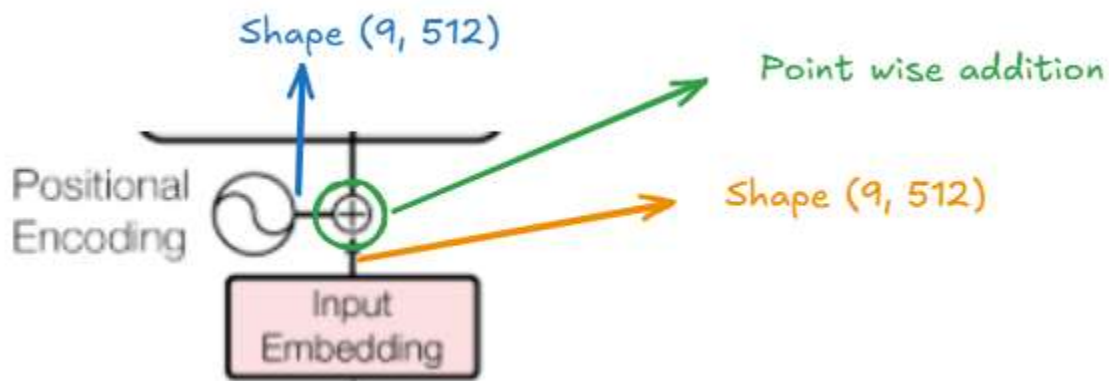
This helps the model to understand

My name is Ahsan

is different from the sentence

Ahsan is my name

Below Figure represents the shapes of the output of **Input Embedding Layer** and the output of **Positional Encoding Layer** for point-wise addition are the same.



Transformer use *sine* and *cosine* functions for positional encoding

For even indices

For odd indices

$$PE_{(pos, 2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right), \quad PE_{(pos, 2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

Where:

- pos = token position
- i = embedding dimension index
- d_{model} = embedding size

Let's create Positional Encoding Vector for Token at position 1 i.e. "name"

Position 1 (Second Token)

Now calculate for $pos=1$.

The denominator values are:

i	Formula	Value
0	$10000^{(0/8)}$	1
1	$10000^{(2/8)}$	10
2	$10000^{(4/8)}$	100
3	$10000^{(6/8)}$	1000

$$\text{Dimension 0} \rightarrow \sin(1/1) = 0.8415$$

$$\text{Dimension 1} \rightarrow \cos(1/1) = 0.5403$$

$$\text{Dimension 2} \rightarrow \sin(1/10) = 0.0998$$

$$\text{Dimension 3} \rightarrow \cos(1/10) = 0.9950$$

$$\text{Dimension 4} \rightarrow \sin(1/100) = 0.0100$$

$$\text{Dimension 5} \rightarrow \cos(1/100) = 0.9999$$

$$\text{Dimension 6} \rightarrow \sin(1/1000) = 0.0010$$

$$\text{Dimension 7} \rightarrow \cos(1/1000) = 0.999999$$

$PE(1) =$

$[0.8415, 0.5403, 0.0998, 0.9950, 0.0100, 0.9999, 0.0010, 0.999999]$

For each token, the **positional encoding** vectors are computed. Thus the output of **Positional Encoding** contains the shape $(9, 8) \rightarrow \text{if } d_{model} = 8$

For $d_{model} = 512 \rightarrow PE \text{ output shape } (9, 512)$

Below figure shows the **Embedding and Position Encoding matrices**, which are then added point-wise.

Input Embedding Matrix (9, 512)

Each row corresponds to a token embedding vector of dimension 512

Token ID	Embedding Dimensions (512)									
	0	1	2	3	4	...	508	509	510	511
101	0.1987	-0.0452	0.6521	-1.0763	0.5347	...	0.1278	-0.9211	0.3312	-0.4136
9	-0.2314	0.8647	-0.3178	0.2315	-0.1023	...	0.6150	0.0746	-0.6354	0.2458
18	0.4929	-0.6236	0.1569	0.7843	-0.5442	...	-0.3211	0.8503	0.1310	-0.7265
20	-0.1187	0.3570	0.9124	-0.2681	0.1896	...	0.4754	-0.1226	0.6788	0.0501
8	0.2716	-0.9032	0.4322	0.3778	-0.7196	...	-0.5338	0.2622	-0.0917	0.3187
200	-0.6213	0.1246	-0.7814	0.2251	0.3128	...	0.0286	-0.7357	0.4256	-0.2472
202	0.3312	0.6085	-0.2241	-0.4716	0.8523	...	-0.1130	0.6442	-0.5224	0.1379
300	-0.2733	-0.3325	0.6741	0.9186	-0.1520	...	0.7091	-0.4063	0.0348	-0.8915
1012	0.4150	-0.1583	-0.5127	0.6302	0.2471	...	-0.3294	0.1787	-0.6101	0.7556

Shape: (9, 512) (9 tokens, 512 embedding dimensions)

Positional Encoding Matrix (9, 512)

Each row corresponds to the positional encoding vector of dimension 512

Position (Index)	Positional Encoding Dimensions (512)									
	0	1	2	3	4	...	508	509	510	511
0	0.0000	1.0000	0.0000	1.0000	0.0000	...	0.0000	1.0000	0.0000	1.0000
1	0.8415	0.5403	0.0998	0.9950	0.0100	...	0.0038	0.9999	0.0004	1.0000
2	0.9093	-0.4161	0.1987	0.9801	0.0200	...	0.0076	0.9999	0.0008	1.0000
3	0.1411	-0.9900	0.2955	0.9553	0.0300	...	0.0114	0.9999	0.0011	1.0000
4	-0.7568	-0.6536	0.3894	0.9211	0.0400	...	0.0152	0.9999	0.0015	1.0000
5	-0.9589	0.2837	0.4794	0.8776	0.0500	...	0.0190	0.9998	0.0019	1.0000
6	-0.2794	0.9602	0.5646	0.8253	0.0600	...	0.0228	0.9997	0.0023	1.0000
7	0.6570	0.7539	0.6442	0.7648	0.0699	...	0.0266	0.9996	0.0026	1.0000
8	0.9894	-0.1455	0.7174	0.6967	0.0799	...	0.0304	0.9995	0.0030	1.0000

Shape: (9, 512) (9 tokens, 512 positional encoding dimensions)

Note:

- The input embedding matrix is learned during training.
- The positional encoding matrix is generated using sine and cosine functions (fixed).
- These two matrices are added element-wise to obtain the final input representation.

Step 3: Encoder

As I have mentioned earlier, that the **encoder** learns the relationship between each token in the input sequence. The relationship is learned with the help of **Multi-Head Attention layer**, which requires three metrices named as **Query (Q)**, **Key (K)**, and **Value (V)**.

- a. **Query (Q)** --> Each token ask question "What kinds of words should I pay attention to?"
- b. **Key (K)** --> Each token says "This is what I am about"
- c. **Value (V)** --> "If you attend me, this is the information you will get"

Q, K, V are computed using weight metrices **W_q**, **W_k** and **W_v**

$$W_q \quad (d_{model}, d_k)$$

$$W_k \quad (d_{model}, d_k)$$

$$W_v \quad (d_{model}, d_v)$$

where,

$$d_k = \frac{d_{model}}{\text{no. of heads in attention layer}} = \frac{512}{8} = 64$$

$$Q = XW_q \quad | \quad K = XW_k \quad | \quad V = XW_v$$

The **Q, K and V** is calculated for each token

For example, calculating the **Q, K and V** for “is” token

$$x_{is} \rightarrow [0.9, 0.8, 0.6, 1]$$

$$Q_{is} = x_{is} * W_q$$

$$K_{is} = x_{is} * W_k$$

$$V_{is} = x_{is} * W_v$$

Since the input matrix **X** contains shape (9,512) and **W_q, W_k, W_v** have shape (512, **dk**).

So the **Q, K, V** have shape (9, **dk**)

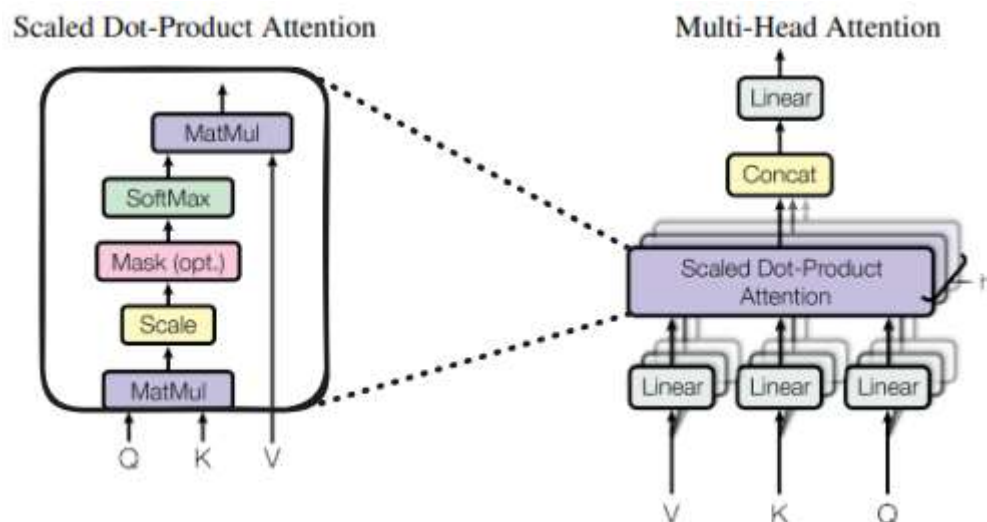
$$Q \rightarrow (9, dk) \rightarrow (9, 64)$$

$$K \rightarrow (9, dk) \rightarrow (9, 64)$$

$$V \rightarrow (9, dk) \rightarrow (9, 64)$$

a. Scaled Dot-Product Attention

The relationship between each token is calculated by **Scaled Dot Product attention** mechanism, which calculated the **attention scores** with the help of **Q and K** metrices.

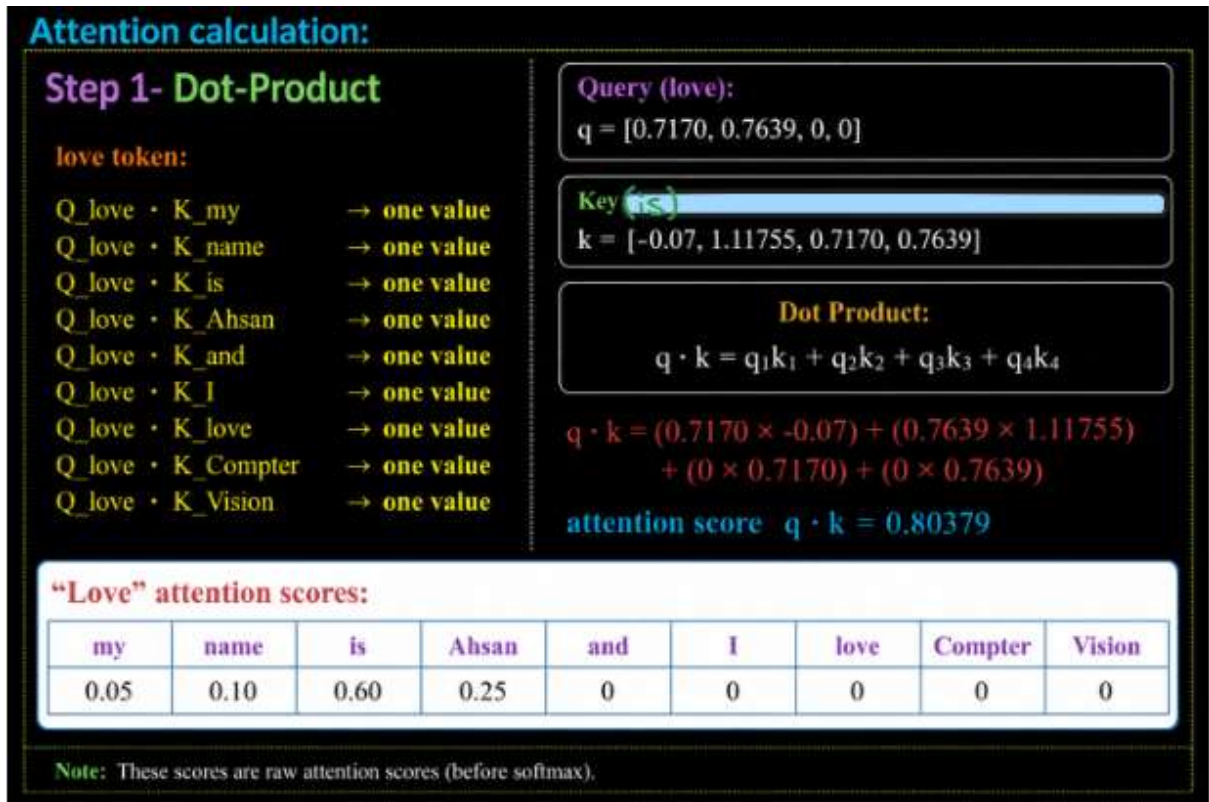


The attention score is calculated by multiplying each **Query** with all the **Keys** – **with the help of MatMul Layer**. The formula of calculating attention score is given below.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The $Q.K^T$ part (MatMul)

Below figure show the **Query** of *Love* token is taken dot-product with the **Keys** of all tokens.



Below figure shows the resulting matrices, after **Q and K** are processed through **MatMul** Layer.

How Transformer Attention Scores are Calculated

Attention calculation:

Row = Query (from)
Column = Key (to)

Tokens (in order):

"my", "name", "is", "Ahsan", "and", "I", "love", "computer", "vision"
101 223 31 987 45 12 78 5561 7752

Full attention Score Matrix									
Query/Key	my	name	is	Ahsan	and	:	love	computer	vision
my	$Q_{my} \cdot K_{my}$	$Q_{my} \cdot K_{name}$	$Q_{my} \cdot K_{is}$	$Q_{my} \cdot K_{Ahsan}$	$Q_{my} \cdot K_{and}$	$Q_{my} \cdot K_{:}$	$Q_{my} \cdot K_{love}$	$Q_{my} \cdot K_{computer}$	$Q_{my} \cdot K_{vision}$
name	$Q_{name} \cdot K_{my}$	$Q_{name} \cdot K_{name}$	$Q_{name} \cdot K_{is}$	$Q_{name} \cdot K_{Ahsan}$	$Q_{name} \cdot K_{and}$	$Q_{name} \cdot K_{:}$	$Q_{name} \cdot K_{love}$	$Q_{name} \cdot K_{computer}$	$Q_{name} \cdot K_{vision}$
is	$Q_{is} \cdot K_{my}$	$Q_{is} \cdot K_{name}$	$Q_{is} \cdot K_{is}$	$Q_{is} \cdot K_{Ahsan}$	$Q_{is} \cdot K_{and}$	$Q_{is} \cdot K_{:}$	$Q_{is} \cdot K_{love}$	$Q_{is} \cdot K_{computer}$	$Q_{is} \cdot K_{vision}$
Ahsan	$Q_{Ahsan} \cdot K_{my}$	$Q_{Ahsan} \cdot K_{name}$	$Q_{Ahsan} \cdot K_{is}$	$Q_{Ahsan} \cdot K_{Ahsan}$	$Q_{Ahsan} \cdot K_{and}$	$Q_{Ahsan} \cdot K_{:}$	$Q_{Ahsan} \cdot K_{love}$	$Q_{Ahsan} \cdot K_{computer}$	$Q_{Ahsan} \cdot K_{vision}$
and	$Q_{and} \cdot K_{my}$	$Q_{and} \cdot K_{name}$	$Q_{and} \cdot K_{is}$	$Q_{and} \cdot K_{Ahsan}$	$Q_{and} \cdot K_{and}$	$Q_{and} \cdot K_{:}$	$Q_{and} \cdot K_{love}$	$Q_{and} \cdot K_{computer}$	$Q_{and} \cdot K_{vision}$
:	$Q_{:} \cdot K_{my}$	$Q_{:} \cdot K_{name}$	$Q_{:} \cdot K_{is}$	$Q_{:} \cdot K_{Ahsan}$	$Q_{:} \cdot K_{and}$	$Q_{:} \cdot K_{:}$	$Q_{:} \cdot K_{love}$	$Q_{:} \cdot K_{computer}$	$Q_{:} \cdot K_{vision}$
love	$Q_{love} \cdot K_{my}$	$Q_{love} \cdot K_{name}$	$Q_{love} \cdot K_{is}$	$Q_{love} \cdot K_{Ahsan}$	$Q_{love} \cdot K_{and}$	$Q_{love} \cdot K_{:}$	$Q_{love} \cdot K_{love}$	$Q_{love} \cdot K_{computer}$	$Q_{love} \cdot K_{vision}$
computer	$Q_{computer} \cdot K_{my}$	$Q_{computer} \cdot K_{name}$	$Q_{computer} \cdot K_{is}$	$Q_{computer} \cdot K_{Ahsan}$	$Q_{computer} \cdot K_{and}$	$Q_{computer} \cdot K_{:}$	$Q_{computer} \cdot K_{love}$	$Q_{computer} \cdot K_{computer}$	$Q_{computer} \cdot K_{vision}$
vision	$Q_{vision} \cdot K_{my}$	$Q_{vision} \cdot K_{name}$	$Q_{vision} \cdot K_{is}$	$Q_{vision} \cdot K_{Ahsan}$	$Q_{vision} \cdot K_{and}$	$Q_{vision} \cdot K_{:}$	$Q_{vision} \cdot K_{love}$	$Q_{vision} \cdot K_{computer}$	$Q_{vision} \cdot K_{vision}$

$$Q \times K^T$$

$$(9,64) \times (64,9)$$

$$\text{Result: } (9,9)$$

The $\frac{1}{\sqrt{d_k}}$ part (Scale)

Each value in the $Q \cdot K^T$ matrix is scaled down by dividing by $\sqrt{d_k}$

The Softmax part:

The softmax function converts each value in the $Q \cdot K^T$ matrix to probability scores

For example, attention scores for *love*

[0.05, 0.1, 0.6, 0.25, 0, 0, 0, 0, 0]

The Final MatMul Layer

$$\text{Output} = \text{softmax}(QK^T / \sqrt{d_k}) \times V$$

$(9, 9)$ $(9, 64)$

Result: $(9, 64)$

The final output matrix of Scaled Dot-Product Attention Layer

Token	Output Vector (64-D)
My	+ [0.32, 0.18, -0.07, 0.94, 0.21, -0.11, 0.56, 0.09, ..., 0.44]
Name	+ [0.41, 0.27, -0.15, 1.02, 0.34, -0.08, 0.61, 0.12, ..., 0.38]
Is	+ [0.55, 0.33, -0.02, 1.10, 0.49, -0.05, 0.72, 0.18, ..., 0.51]
Ahsan	+ [0.60, 0.42, -0.21, 1.25, 0.57, -0.14, 0.80, 0.22, ..., 0.66]
And	+ [0.29, 0.20, -0.10, 0.88, 0.31, -0.09, 0.52, 0.15, ..., 0.40]
I	+ [0.46, 0.38, -0.18, 1.15, 0.53, -0.12, 0.74, 0.25, ..., 0.59]
Love	+ [0.70, 0.55, -0.05, 1.42, 0.68, -0.03, 0.91, 0.30, ..., 0.78]
Computer	+ [0.66, 0.51, -0.09, 1.35, 0.62, -0.06, 0.88, 0.28, ..., 0.73]
Vision	+ [0.72, 0.58, -0.04, 1.48, 0.71, -0.02, 0.95, 0.33, ..., 0.81]

Shape
(9, 64)

b. MultiHead Attention:

In the Multi-Head Attention layer, multiple Scaled Dot-Product Attention operations run in parallel.

Each parallel attention unit is called a **Head**.

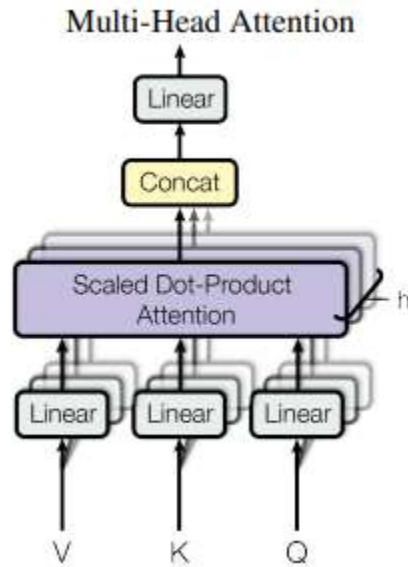
Different heads learn different types of relationships between tokens, such as:

- semantic relationships

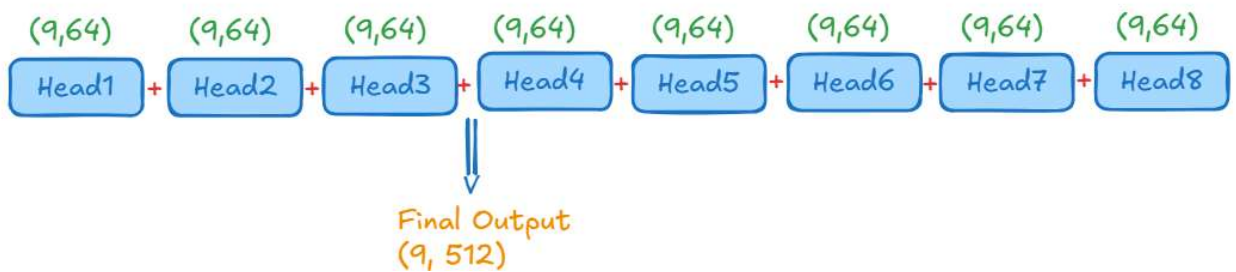
- grammatical structure
- long-range dependencies
- contextual meaning

For example:

- one head may focus on subject-object relationships
- another head may focus on nearby words
- another head may capture semantic similarity



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$



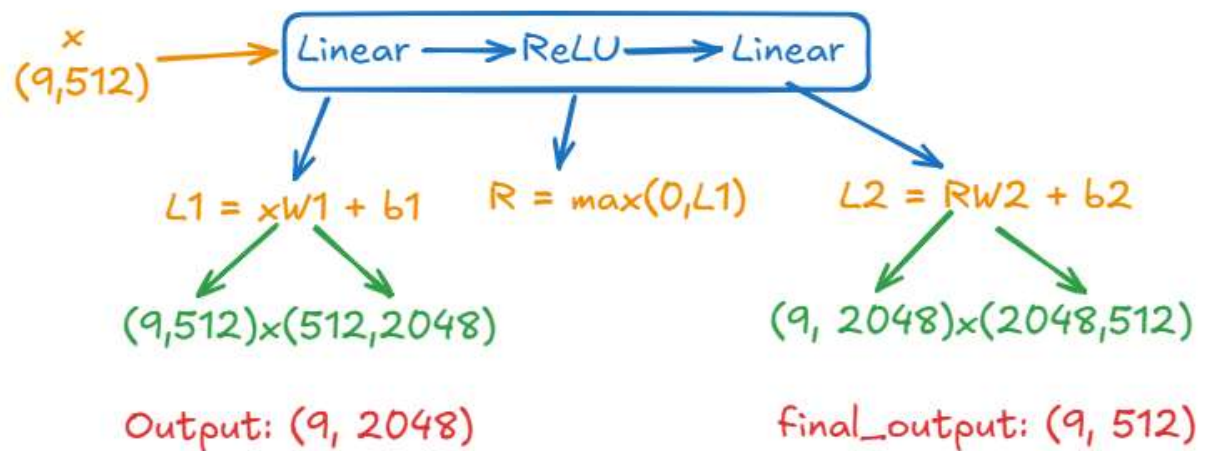
c. Add and Norm Layer

$$\text{Output1} = \text{LayerNorm}(X + \text{Attention_Output})$$

$\downarrow$ $\downarrow$
9, 512 9, 512

d. Feed Forward Layer

The feed forward layer consists of 3 layers

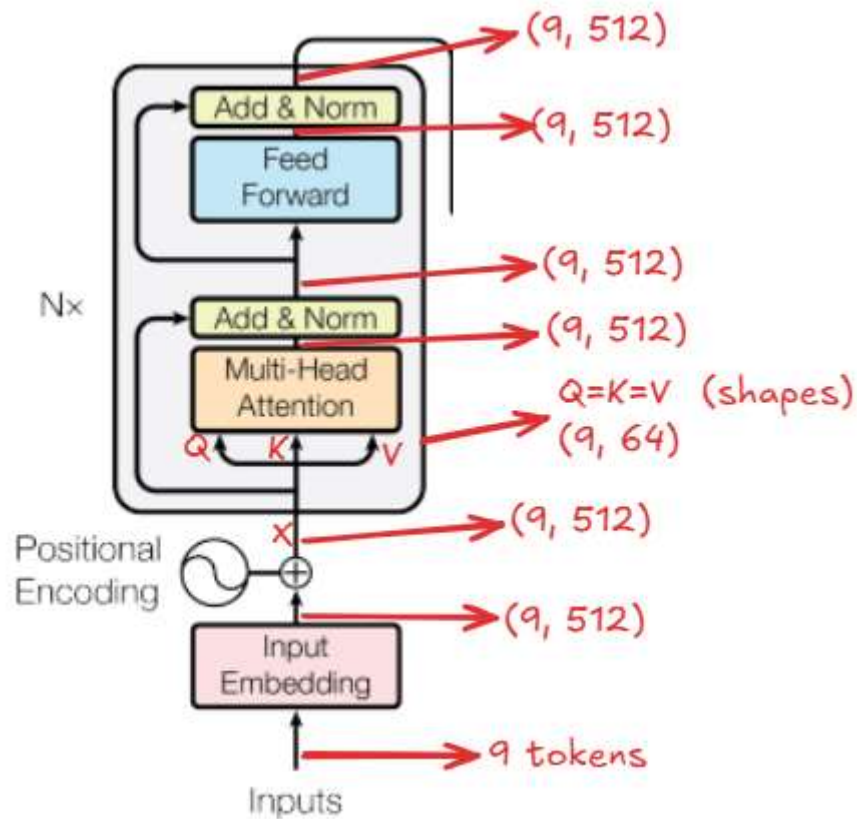


e. Final add and Norm Layer

$$\text{final_Output} = \text{LayerNorm}(\text{Output1} + \text{Feedforward_Output})$$

$\downarrow$ $\downarrow$ $\downarrow$
9, 512 9, 512 9, 512

Below figure show the shapes of metrics calculated at each layer in the Encoder



In original Transformers research paper “Attention is all you need” – there are 6 encoders

